

Laporan Praktikum 4 Kontrol Cerdas

Nama : Levis Mega Nureska

NIM 224308010

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/levismeganureska>

Student Lab Assistant : Muhammad Fatih Anfa Adhima

1. Judul Percobaan

Implementasi Reinforcement Learning untuk Pengendalian Otonom dalam Lingkungan Simulasi

2. Tujuan Percobaan

Tujuan dari percobaan praktikum Reinforcement Learning yaitu:

1. Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
2. Mengimplemtasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
3. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom

3. Landasan Teori

Reinforcement Learning adalah proses mapping (pemetaan) dari situasi yang ada di environment (states) ke bentuk aksi (behavior) agar dapat memaksimalkan reward. Agent yang bertindak sebagai sang learner tidak perlu diberitahukan behavior apakah yang akan sepatutnya dilakukan, atau dengan kata lain, sang learner belajar sendiri dari pengalamannya. Salah satu konsep utama dalam Reinforcement Learning yaitu Agent. Dalam Reinforcement Learning (RL), Agent adalah entitas atau program yang mengambil keputusan dan berinteraksi dengan lingkungan untuk mencapai tujuan tertentu. Agent mengambil tindakan, mendapatkan feedback, dan belajar dari pengalaman untuk meningkatkan performanya. Tugas utama dari Agent yaitu mengamati lingkungan, memilih tindakan atau action, menerima feedback, dan memperbarui strategi. Maka dari itu Agent adalah inti dari Reinforcement Learning yang bertindak berdasarkan pengamatan lingkungan, menerima reward, dan terus belajar untuk meningkatkan kemampuannya.

Deep Q-Network (DQN) adalah algoritma dalam reinforcement learning yang menggabungkan Q-learning dengan jaringan saraf dalam untuk menyelesaikan masalah kompleks, sehingga memungkinkan agen untuk belajar dari pengalaman dan membuat keputusan dalam lingkungan yang rumit. Algoritma ini sangat berguna untuk menangani masalah dengan ruang keadaan yang besar, seperti pengenalan gambar dan kontrol robot, dengan memanfaatkan kemampuan jaringan saraf untuk mengekstrak fitur dari data secara efisien dan efektif. DQN bekerja dengan cara memanfaatkan jaringan saraf untuk memperkirakan nilai Q, yang merupakan nilai dari setiap aksi yang mungkin diambil dalam

suatu keadaan tertentu. Dengan menggunakan pemutaran ulang pengalaman, DQN menyimpan pengalaman masa lalu dalam memori dan mengambil sampel acak dari pengalaman tersebut untuk memperbarui model, sehingga meningkatkan stabilitas dan efisiensi pembelajaran. Selain itu, DQN juga menerapkan strategi eksplorasi epsilon-greedy, yang memungkinkan agen untuk menyeimbangkan antara eksplorasi tindakan baru dan eksploitasi tindakan yang sudah diketahui, sehingga agen dapat menemukan strategi yang optimal seiring waktu.

4. Analisis dan Diskusi

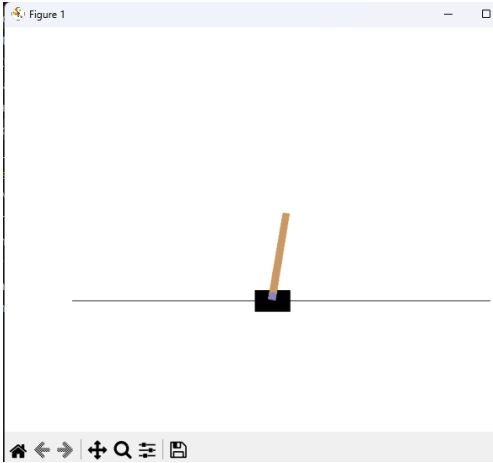
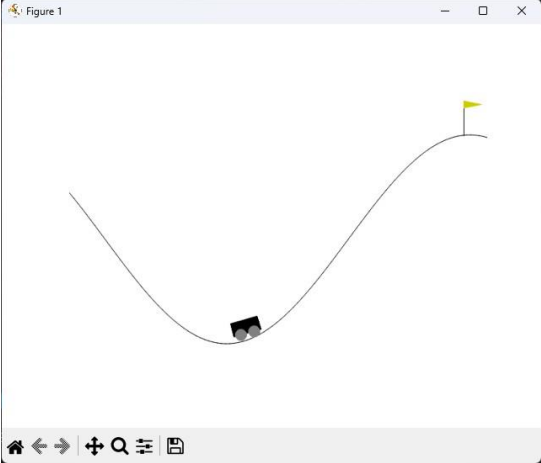
MountainCar-v0 adalah sebuah lingkungan simulasi di mana sebuah mobil harus mencapai puncak bukit, meskipun tenaga mesinnya tidak cukup untuk langsung mencapainya. Lingkungan ini memiliki dua variabel keadaan, yaitu posisi mobil di sepanjang bukit dan kecepatannya. Agen dapat memilih salah satu dari tiga aksi yang tersedia: bergerak ke kiri, tetap di tempat, atau bergerak ke kanan. Untuk mencapai tujuan, agen harus menentukan strategi yang tepat agar mobil dapat memperoleh momentum yang cukup. Dalam simulasi ini, agen mendapatkan penalti sebesar -1 untuk setiap langkah yang diambil, sehingga mendorongnya untuk mencapai puncak secepat mungkin. Dalam beberapa versi, sistem reward ini dapat dimodifikasi guna mempercepat proses pembelajaran. Tantangan utamanya adalah bahwa mobil perlu berayun bolak-balik untuk mengumpulkan energi yang cukup agar dapat mencapai puncak. Reinforcement Learning (RL) digunakan untuk melatih agen dalam lingkungan ini. Agen belajar melalui interaksi langsung dengan lingkungan, mencoba berbagai aksi, dan menerima feedback dalam bentuk reward atau penalti. Dengan menerapkan algoritma RL, agen dapat mengembangkan strategi optimal untuk menyelesaikan tugas, belajar dari pengalaman sebelumnya, dan beradaptasi dengan perubahan kondisi. Mengingat bahwa lingkungan ini memiliki ruang keadaan yang kontinu, pendekatan menggunakan Deep Q-Network (DQN) menjadi pilihan yang efektif. DQN memanfaatkan jaringan saraf untuk memperkirakan nilai Q, sehingga agen dapat belajar lebih efisien dan meningkatkan kemampuannya dalam mencapai puncak bukit.

5. Assignment

Pada praktikum ini, algoritma Deep Q-Network (DQN) diterapkan untuk melatih agen dalam lingkungan MountainCar-v0. Tujuan utama agen adalah mencapai puncak bukit dengan memilih aksi yang paling optimal. Implementasi kode dimulai dengan mengimpor berbagai pustaka yang diperlukan, seperti **gymnasium** untuk mengelola lingkungan reinforcement learning, **numpy** untuk operasi numerik, **random** untuk pemilihan aksi secara acak, serta **deque** dari collections untuk menyimpan pengalaman agen. Selain itu, **tensorflow.keras** digunakan untuk membangun dan melatih model jaringan saraf. Agen DQN didefinisikan dalam sebuah kelas bernama **DQNAgent**, yang diinisialisasi berdasarkan ukuran keadaan dan jumlah aksi yang tersedia. Beberapa parameter utama yang digunakan dalam algoritma ini meliputi **gamma** sebagai faktor diskon untuk reward di masa depan, **epsilon** untuk mekanisme eksplorasi, serta **learning_rate** yang menentukan kecepatan optimasi model. Pada eksperimen pertama, yaitu **CartPole-V1**, agen bertugas menjaga keseimbangan tiang di atas gerobak. Karena lingkungan ini relatif sederhana dan lebih dapat diprediksi, agen dapat belajar dengan cepat. Setiap kali berhasil menjaga tiang tetap tegak, agen mendapatkan reward positif. Pelatihan untuk CartPole dapat selesai dalam beberapa ratus episode karena agen dengan mudah mencapai performa optimal. Sebaliknya, dalam **MountainCar-v0**, tantangan yang dihadapi lebih kompleks. Agen harus menggerakkan mobil ke puncak bukit, tetapi tenaga mobil tidak cukup untuk langsung mencapai tujuan. Oleh karena itu, agen perlu membangun momentum dengan bergerak maju dan mundur secara strategis. Reward dalam lingkungan ini lebih sulit diperoleh, karena agen sering kali gagal mencapai puncak bukit di awal pembelajaran. Dikarenakan tingkat kesulitannya yang lebih tinggi,

MountainCar-v0 memerlukan arsitektur model yang lebih kompleks atau teknik tambahan untuk meningkatkan stabilitas pelatihan. Proses belajar dalam lingkungan ini juga memerlukan lebih banyak episode sebelum agen dapat mencapai performa optimal. Selain itu, nilai reward yang diperoleh bisa sangat fluktuatif, sehingga diperlukan lebih banyak penyesuaian agar agen dapat mencapai skor yang lebih tinggi secara konsisten.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan	
		Cartpole	Mountain car
1	Hasil Score (DQN)	Episode: 9/20, Score: 11, Epsilon: 0.93 Episode: 10/20, Score: 21, Epsilon: 0.92 Episode: 11/20, Score: 29, Epsilon: 0.91 Target Network updated! Episode: 12/20, Score: 19, Epsilon: 0.90 Episode: 13/20, Score: 20, Epsilon: 0.90 Episode: 14/20, Score: 11, Epsilon: 0.89 Episode: 15/20, Score: 25, Epsilon: 0.88 Episode: 16/20, Score: 8, Epsilon: 0.87 Episode: 17/20, Score: 23, Epsilon: 0.86 Episode: 18/20, Score: 8, Epsilon: 0.85 Episode: 19/20, Score: 32, Epsilon: 0.84 Episode: 20/20, Score: 18, Epsilon: 0.83	Target Network updated! Episode: 2/20, Score: 199, Epsilon: 0.99 Episode: 3/20, Score: 199, Epsilon: 0.99 Episode: 4/20, Score: 199, Epsilon: 0.99 Episode: 5/20, Score: 199, Epsilon: 0.98 Episode: 6/20, Score: 199, Epsilon: 0.98 Episode: 7/20, Score: 199, Epsilon: 0.97 Episode: 8/20, Score: 199, Epsilon: 0.97 Episode: 9/20, Score: 199, Epsilon: 0.96 Episode: 10/20, Score: 199, Epsilon: 0.96 Episode: 11/20, Score: 199, Epsilon: 0.95
2	Hasil Score (Test)	Test Episode: 1, Score: 23 Test Episode: 2, Score: 15 Test Episode: 3, Score: 18 Test Episode: 4, Score: 22 Test Episode: 5, Score: 53 Test Episode: 6, Score: 31 Test Episode: 7, Score: 16 Test Episode: 8, Score: 23 Test Episode: 9, Score: 20 Test Episode: 10, Score: 23	Test Episode: 1, Score: 199 Test Episode: 2, Score: 199 Test Episode: 3, Score: 199 Test Episode: 4, Score: 199 Test Episode: 5, Score: 199 Test Episode: 6, Score: 199 Test Episode: 7, Score: 199 Test Episode: 8, Score: 199 Test Episode: 9, Score: 199 Test Episode: 10, Score: 199
3	Hasil Run		

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

1. Penggunaan target network membantu stabilitas pembelajaran, menghindari perubahan nilai Q yang terlalu drastis.
2. MountainCar-v0 lebih sulit dipelajari karena *reward* hanya diberikan saat mencapai puncak bukit, memerlukan strategi eksplorasi lebih lama, dan mengandalkan momentum untuk berhasil. Sedagkan Cartpole lebih cepat karena agen dapat menerima reward setiap Langkah yang menjadikannya lebih optimal

8. Saran

Untuk meningkatkan agar performa agen DQN lebih baik bisa menerapkan *Target Network* untuk meningkatkan stabilitas dan mencegah model dari *overfitting* terhadap pola jangka pendek, dengan cara memperbarui target *network* secara periodik. strategi epsilon *decay* yang lebih adaptif, seperti menyesuaikannya berdasarkan performa, dapat memperbaiki keseimbangan eksplorasi dan eksploitasi. dapat mengoptimalkan *hyperparameter* seperti gamma, learning rate, dan epsilon

9. Daftar Pustaka

Angelina, L., Purwanto, Y., & Novianty, A. (n.d.). *Pengambilan Keputusan Pada Trafik Management Dengan Menggunakan Reinforcement Learning Decision Making on Traffic Management Using Reinforcement Learning*

Baradja, A., & Tjendrowasono, T. I. (2024). Pengaplikasian Deep Reinforcement Q-Learning Untuk Prediksi Perdagangan Valas Otomatis. *Jurnal Rekayasa Sistem Informasi dan Teknologi*, 1(3), 190–198. <https://doi.org/10.59407/jrsit.v1i3.519>